

EX- PRACTICAL THEORETICAL

Jun 2018 row 2

for sums/concat $\rightarrow T_{result} = T\{ \}$
 max/min $\rightarrow T_{result} = v[0]$
 product $\rightarrow$

```
1. void testFct()
{
    vector<int> v1 {4, 2, 1, -4};
    assert(fct<int>(v1) == 3);
    vector<int> v2;
    try {
        fct<int>(v2);
        assert(false); // empty vector
    }
    catch (std::exception&) {
        assert(true);
    }
    vector<double> v3 {2, 10.5, 5, -10};
    assert(fct<double>(v3) == 7.5);
    vector<string> v4 {"y", "g", "a", "u"};
    assert(fct<string>(v4) == "ygam");
}
```

(SUM)

```
 $\rightarrow$  template <typename T>
T fct(const vector<T>& v)
{
    if(v.empty())
        throw exception();
    T sum = v[0];
    for(int i = 1; i < v.size(); i++)
        sum = sum + v[i];
    return sum;
}
```

(MAX ELEM)

```
 $\rightarrow$  template <typename T>
T fct(vector<T> v)
{
    if(v.empty())
        throw exception();
    return *max_element(begin(v),
        end(v));
}
```

Jun 2018 -1

```
1. void testFct()
{
    vector<int> v1 {4, 2, 1, 6, 3, -4};
    assert(fct<int>(v1) == 6);
    vector<int> v2;
    try {
        ...
    }
    catch...
    vector<double> v3 {2, 10.5, 6.33, -100, 9, 1};
    assert(fct<double>(v3) == 10.5);
}
```

MAX ELEM (easier)

```
 $\rightarrow$  template <typename T>
T fct(const std::vector<T>& v)
{
    if(v.empty())
        throw std::exception();
    T max = v[0];
    for(const T& elem : v)
        if(elem > max)
            max = elem;
    return max;
}
```

$\rightarrow$ SUM (easier)

```
template <typename T>
T fct(const std::vector<T>& v)
{
    if(v.empty())
        throw std::exception();
    T result = T{0};
    for(const T& elem : v)
        result += elem;
    return result;
}
```



```

1. void function2()
{
    Adder<int> add(5);
    add = add + 5 + 2;
    ++add;
    add + 8;
    cout << add.sum() << "\n";
    --add;
    cout << add.sum() ...
    --(--add);
    cout << add.sum() << ...
    try {
        --(--(--add));
    }
    catch (runtime_error& ex)
    {
        cout << ex.what();
    }
}

```

```

→ template <typename T>
class Adder {
private:
    vector<T> values;

```

```

public:
    Adder(T value) {
        values.push_back(value);
    }
    Adder& operator+ (T value) {
        values.push_back(value);
        return *this;
    }
    Adder& operator++ () {
        values.push_back(values.back());
        return *this;
    }

```

```

    Adder& operator-- () {
        if (values.size() == 1)
            throw runtime_error("No more");
        values.pop_back();
        return *this;
    }
    T sum() const {
        T s = values[0];
        for (int i = 1; i < values.size(); i++)
            s = s + values[i];
        return s;
    }
}

```

TEMPLATES:

Stack<string> s{2}; ⇒ template <typename T>
class Stack

Activity a{...}; ⇒ class Activity

fct<int>(v) ⇒ template <typename T>
T fct(...)

Activity("dexa", "time") ⇒ data members
s.getMaxCapacity() ⇒ int maxCapacity
todo += activity ⇒ vector<T> elems

Activity a{4x4, "y"} ⇒

⇒ Activity(const string&, const string&)

SmartPointer<int> p{new int{33}} ⇒ SmartPointer(T*)

Stack<string> s{2} ⇒ Stack(int)

u) s = s + elem ⇒ operator+

todo += activity ⇒ operator+=

cout << a ⇒ friend ostream& operator<<(...)

if (current == elem) ⇒ operator==
set + duplicate →

cout << *e ⇒ operator*()

--add ⇒ operator--()

++add ⇒ operator++()

1) what classes?

2) what data stored? Data members?

3) what constructors? Default?
Copy?

4) what operators overload?

5) try/catch?

6) range-based for?

7) output printed?

8) inheritance/polymorphism?

9) dynamic alloc? destructors?

10) emphasized operation? 2

- "row 2"

real, imaginary
(private fields)

```
void Complex() {
```

```
Complex a{}; b{1,2}; c{6,4}; d{6};
```

```
assert(a.getReal() == 0 && a.getImaginary() == 0);
```

```
assert(c.getImaginary() == 4);
```

```
assert(b == d); → const Complex & other
```

```
Complex res1 = c / 2; → operator / (int value)
```

```
cout << res1 << " * m"; → operator <<
```

```
try {
```

```
Complex res2 = b / 0;
```

```
}
```

```
catch (runtime_error & e) {
```

```
assert ...
```

```
}
```

template <type name T> class Vector

```
Vector<string> v1{std::vector<string>{"hello", "bye"}};
```

```
v1.printAll(std::cout);
```

```
Vector<string> v2{std::vector<Complex>{a, b, c, d}};
```

```
v2.printAll(std::cout);
```

```
}
```

```
class Complex {
```

```
private:
```

```
int real;
```

```
int imaginary;
```

```
public:
```

```
Complex(): real{0}, imaginary{0} {}
```

```
Complex(int real, int imaginary): real{real}, imaginary{imaginary} {}
```

```
int getReal() const {
```

```
return this->real;
```

```
} int getImaginary() const {
```

```
return this->imaginary;
```

```
bool operator==(const Complex & other) const {
```

```
return this->real == other.real &&
```

```
this->imaginary == other.imaginary;
```

```
}
```

```
Complex operator/(int value) const {
```

```
if (value == 0)
```

```
throw runtime_error{"Division by zero"};
```

```
return Complex{this->real / value, this->imaginary / value};
```

```
}
```

```
friend ostream& operator<<(ostream& out, const Complex & c);
```

```
}
```

```
ostream& operator<<(ostream& out, const Complex & c) {
```

```
out << c.real << "+" << c.imaginary << "i";
```

```
return out;
```



```
template <typename T>
```

```
class Vector {
```

```
private:
```

```
    vector<T> elems;
```

```
public:
```

```
    Vector(const vector<T>& elems): elems{elems} {}
```

```
    void printAll(ostream& out) const {
```

```
        for (const auto& elem: elems)
```

```
            out << elem << " ";
```

```
        out << '\n';
```

```
    }
```

2020 covid

```
1. void fct1() {
```

```
    HTML_Element* p1 = new HTMLParagraph { "Exam" };
```

```
    : ... = new HTMLImage ...
```

```
}
```

```
class HTML_Element {
```

```
public:
```

```
    virtual std::string getHTMLString() const = 0;
```

```
    virtual ~HTML_Element() = default;
```

```
};
```

```
class HTMLParagraph: public HTML_Element {
```

```
private:
```

```
    std::string text;
```

```
public:
```

```
    HTMLParagraph(const std::string& text): text{text} {}
```

```
    std::string getHTMLString() const override {
```

```
        return "<p>" + text + "</p>";
```

```
    }
```

```
};
```

```
class HTMLImage: public HTML_Element {
```

```
private:
```

```
    std::string source;
```

```
public:
```

```
    HTMLImage(const std::string& source): source{source} {}
```

```
    std::string getHTMLString() const override {
```

```
        return "<img>" + source + "</img>";
```

```
    }
```

```
};
```


template <typename T>

class HTMLBuilder {

private:

vector<T> elems;

public:

HTMLBuilder & operator += (T elem) {

if (elem == nullptr)

throw runtime_error {"cannot"};

elems.push_back(elem);

return *this;

}

friend ostream & operator << (ostream & out, const HTMLBuilder<T> & html)

{ out << "<html><body>";

for (const auto & elem: html.elems)

out << elem->getHTMLString();

out << "</html><body>";

return out;

}

};

Smart pointers, vector.

4.1. jpeg - Row 1

void function1() {

SmartPointer<int> i1 {new int {1}};

SmartPointer<int> i2 {new int {2}};

SmartPointer<int> i3 {new int {3}};

Vector<SmartPointer<int>> v1{}; → empty custom vect.

v1.add(i1).add(i2).add(i3); → add 3 elems

for (auto e: v1)

cout << *e << " "; // 1, 2, 3

SmartPointer<string> s1 {new string {"A"}};

SmartPointer<string> s2 = s1; → copy constructor

SmartPointer<string> s3 {new string {"C"}};

Vector<SmartPointer<string>> v2{};

v2.add(s2).add(s1);

try {

v2 = v2 - s2;

v2 = v2 - s3;

} catch (...) {}

}

}

→ SmartPointer (T *p)

→ chaining: add must return reference to the same vector

→ T & operator *()


```
template <typename T>
```

```
class SmartPointer {
```

```
private:
```

```
T* ptr; // SmartPointer<int> s1{new ...};
```

```
int* count; // SmartPointer<int> s2 = s1;
```

```
public:
```

```
SmartPointer(T* ptr) : ptr{ptr}, count{new int{1}} {}
```

```
SmartPointer(const SmartPointer<T>& other) : ptr{other.ptr}, count{other.count} {
```

```
    (*count)++;
```

```
}
```

```
~SmartPointer() {
```

```
    (*count)--;
```

```
    if (*count == 0) {
```

```
        delete ptr;
```

```
        delete count;
```

```
    }
```

```
T& operator*() const { // cout < s1;
```

```
    return *ptr; // not the actual int/string value
```

```
}
```

```
bool operator==(const SmartPointer<T>& other) const {
```

```
    return *ptr == *other.ptr;
```

```
};
```

```
template <typename T> // Vector<SmartPointer<int>>, Vector<SmartPointer<string>>
```

```
class Vector {
```

```
private:
```

```
vector<T> elems;
```

```
public:
```

```
Vector() = default; // Vector<SmartPointer<int>> v1{};
```

```
Vector<T>& add(const T& elem) { // v1.add(i1).add(i2).add(i3)...
```

```
    elems.push_back(elem); // store i1, i2, i3, ...
```

```
    return *this; // chaining: .add(...).add(...)
```

```
}
```

```
typename Vector<T>::iterator begin() { // for (auto e: v1)
```

```
    return elems.begin();
```

```
}
```

```
typename Vector<T>::iterator end() {
```

```
    return elems.end();
```

```
}
```

```
Vector<T> operator-(const T& elem) const { // v2 = v2 - s2
```

```
    Vector<T> result;
```

```
    bool found = false;
```

```
    for (const auto& current: elems) {
```

```
        if (!found && current == elem) found = true;
```

```
        else result.add(current);
```

```
}
```

```
    if (!found) throw ...
```

```
    / return result {};
```


row 2

SmartPointer, Set.

```
d function2() {
    SmartPointer<string> s1 {new string { "A" } };
    SmartPointer<string> s2 = s1; → copy constructor
    ...
    Set<SmartPointer<string>> set1 { };
    try {
        Set1 = set1 + s1;
        Set1 = set1 + s2;
    }
    catch ... {
    }
    SmartPointer<int> i1 {new int {1}};
    ... i2 { ... {2}};
    ... i3 { ... {3}};
    Set<SmartPointer<int>> set2 { };
    Set2 = set2 + i1;
    Set2 = set2 + i2;
    Set2 = set2 + i3;
    set2.remove(i1).remove(i3);
    for (auto e: set2)
        cout << *e << " ";
}
```

SmartPointer that owns a dynamically allocated string "A".

```
template<typename T> // SmartPointer<string>, SmartPointer<int>
class SmartPointer {
private:
    T* ptr;
    int* count; // s2 = s1
public:
    SmartPointer(T* ptr): ptr {ptr}, count {new int {1}} // Smart ... <string> s1 {new
    SmartPointer(const SmartPointer<T>& other): ptr {other.ptr}, count {other.count
        (*count)++;
    }
    SmartPointer<T>& operator = (const SmartPointer<T>& other) {
        if (&this == &other) return *this; // protect self-assignment
        (*count)--;
        if (*count == 0) {
            delete ptr; delete count;
        }
        ptr = other.ptr;
        count = other.count;
        (*count)++;
        return *this;
    }
}
```



```

~SmartPointer() {
    (*count)--;
    if (*count == 0) {
        delete ptr;
        delete count;
    }
}

T& operator*() const { // cout << *e
    return *ptr;
}

bool operator==(const SmartPointer<T>& other) const {
    return *ptr == *other.ptr;
}
};

```

```

template <typename T> // Set < SmartPointer<string>> <int>
class Set {
private:
    vector<T> elems;
public:
    Set() = default; // set < SmartPointer<string>> set1 {}
    Set<T> operator+(const T& elem) const {
        Set<T> result = *this; // create copy
        for (const auto& current : result.elems)
            if (current == elem) // set1 = set1 + s2 duplicate case
                throw runtime_error{"Elem .. "};
        result.elems.push_back(elem);
        return result;
    }

    Set<T>& remove(const T& elem) { // set2.remove(i1).remove(i3)
        for (auto it = elems.begin(); it != elems.end(); ++it) { // search
            if (*it == elem) { // compare
                elems.erase(it); // remove
                return *this; // chaining .remove(...).remove(...)
            }
        }
        return *this;
    }

    typename vector<T>::iterator begin() {
        return elems.begin();
    }

    typename vector<T>::iterator end() {
        return elems.end();
    }
};

```


✓ "Exam guide"

```

1. void testStack() {
    Stack<std::string> s{2};
    assert(s.getMaxCapacity() == 2);
    try {
        s = s + "exam";
        s = s + "oop";
        s = s + "test"; // not added because stack is full
    }
    catch (...) {
        assert(...);
    }
    assert(s.pop() == "oop");
    assert(s.pop() == "exam");
}
    
```

→ Stack (int cap)

→ int getMaxCapacity() const

→ Stack<T> operator+ (...) const

→ throw ...

Data members:

```

vector<T> elems;
int maxCapacity;
    
```

```

template <typename T>
class Stack {
private:
    vector<T> elems;
    int maxCapacity;
public:
    Stack(int capacity): maxCapacity {capacity} {}
    int getMaxCapacity() const {
        return maxCapacity;
    }
    Stack<T> operator+(const T& elem) const {
        if (elems.size() == maxCapacity)
            throw ...
        Stack<T> result = *this;
        result.elems.push_back(elem);
        return result;
    }
    T pop() {
        T last = elems.back();
        elems.pop_back();
        return last;
    }
};
    
```


1.

```
void ToDoList() {
```

```
    Todo<Activity> todo{};
```

```
    Activity hff{"go...", "20:00"};
```

```
    todo += hff;
```

```
    Activity project{"present...", "09:20"};
```

```
    todo += project;
```

```
    for (auto a: todo)
```

```
        std::cout << a << '\n';
```

```
    todo.reversePrint(std::cout);
```

```
}
```

template <typename>

class Todo

description; time (data members)

operator +=

```
class Activity {
```

```
private:
```

```
    std::string description;
```

```
    std::string time;
```

```
public:
```

```
    Activity(const std::string& description, const std::string& time):
```

```
        : description{description}, time{time} {}
```

```
    friend ostream& operator<<(ostream& out, const Activity& a) {
```

```
        out << "Activity" << a.description
```

```
        << "will take place at" << a.time
```

```
        << "\n";
```

```
        return out;
```

```
    }
```

```
};
```

```
template <typename T>
```

```
class Todo {
```

```
private:
```

```
    vector<T> elems;
```

```
public:
```

```
    Todo() = default;
```

```
    Todo<T>& operator+=(const T& elem) {
```

```
        elems.push_back(elem);
```

```
        return *this; // chaining
```

```
    }
```

```
    typename vector<T>::iterator begin() {
```

```
        return elems.begin();
```

```
    }
```

```
    typename vector<T>::iterator end() {
```

```
        return elems.end();
```

```
    }
```

```
    void reversePrint(ostream& out) const {
```

```
        for (auto it = elems.rbegin(); it != elems.rend(); ++it) {
```

```
            out << *it << '\n';
```

```
    }
```

```
};
```

```
}
```


Jun 2026 row 2 (915)

```
void function()
{
    Character* c = new Character {'c'};
    assert (*c == 'c');
    MyObjects objects{c}; // starts container with one object
    objects.addBeginning(new Double {2.5})
        • addBeginning(new Double {1.23})
        • addBeginning(new Character {'o'});
    assert(objects.size() == 4);
    for (Object* o : objects) // not the vector(*), but the variable (object) of type MyObjects!
        o->print();
}
```

→ classes: Object (virtual) ←
Character (inherits)
Double (inherits)
MyObjects (n refs)

class Object{

public:

virtual void print() const = 0;

virtual ~Object() = default;

};

class Character: public Object{

private:

char value;

public:

Character(char value): value{value}{};

char operator*() const {

return value;

};

void print() const override {

cout << value << " ";

};

};

class MyObjects{

private:

std::vector<Object*> objects;

public:

MyObjects(Object* object) { // starts container with one object.
objects.push_back(object);

};

MyObjects& addBeginning(Object* object){

objects.insert(objects.begin(), object); // add at the beginning

return *this; // chaining

};

int size() const { return objects.size(); }

auto begin() { return objects.begin(); }

auto end() { return objects.end(); }

};

~MyObjects() { for(auto object: objects) delete object; }

class Double: public Object{

private:

double value;

public:

Double(double value): value{value}{};

void print() const override {

cout << value << " ";

};

};